

说明

CoreMark 是由 EEMBC 于 2009 年推出的嵌入式处理器性能评测标准，旨在取代过时的 Dhrystone，现已成为业界公认的 MCU 性能衡量基准。

CoreMark 通过运行三种核心算法评估处理器效率：矩阵运算（加减乘除及乘向量）、链表操作（排序与 CRC 校验）和状态机迁移（循环与分支控制）。测试代码量约 2KB，数据量 16KB，最终以 Iterations/Sec（每秒迭代次数）作为输出结果，要求测试时长不少于 10 秒以确保准确性。

CoreMark 得分受多种因素影响。编译器优化等级是最关键因素之一，从 -O1 到 -Ofast，性能差异显著。特定优化选项如循环展开（-funroll-all-loops）、函数内联（-finline-limit）和指令调度（-fschedule-selective-scheduling）可进一步提升跑分。此外，不同 GCC 版本也会带来约 1.5% 的性能差异。

内存配置同样至关重要。RAM 可实现零等待访问，而 Flash 读取较慢，高频下需插入等待周期，导致性能下降。将核心代码放入 RAM 执行可显著提升跑分。

适用范围

适用范围	系列
通用 MCU	CH32V307/CH32H417/CH32V205

目录

说明 1

目录 2

表格索引..... 2

图片索引..... 2

第 1 章 一级标题..... 1

 1. 1 二级标题 错误!未定义书签。

 1. 1. 1 三级标题 错误!未定义书签。

第 2 章 一级标题..... 错误!未定义书签。

 2. 1 二级标题 错误!未定义书签。

 2. 1. 1 三级标题 3

 2. 1. 2 三级标题 4

历史版本..... 6

声明 7

表格索引

第一张表格 错误!未定义书签。

图片索引

图片题注..... 错误!未定义书签。

第1章 Coremark 介绍

1.1 Coremark 简介

CoreMark 是用来衡量嵌入式微控制器 MCU 性能的标准。

CoreMark 由 EEMBC(Embedded Microprocessor Benchmark Consortium)于 2009 年提出并开发。用于替换过时的 Dhrystone 标准。目前该标准已经成为测量与比较各类处理器的业界标准测试。Coremark 通过运行一组算法（2K 代码，16K 数据）来评估 MCU 内核的效率。通过单位时间内同一组算法在不同 MCU 上运行的次数，来对比性能，Coremark 得分越高意味着 MCU 性能越好。

1.2 Coremark 测试内容

Coremark 工作负载主要包括三种常用的算法：

矩阵操作：Coremark 中矩阵相关的实现在 core_matrix.c 文件中，在 main 函数中检测到需要进行矩阵测试的时候，调用 core_init_matrix()函数来完成矩阵的初始化操作，然后使用函数 matrix_test()来做测试，具体的测试为：为矩阵所有元素加上一个常数、矩阵乘一个常数、矩阵乘向量、矩阵乘矩阵、再次为矩阵所有元素加一个常数，之后将矩阵元素还原。

列表操作：Coremark 中列表相关的实现在 core_list_join.c 文件中，在 main 函数中检测到需要进行列表测试的时候，会调用 core_list_init()函数，core_list_init()函数会完成列表相关的初始化处理，之后会调用 core_bench_list()函数来完成列表的排序、CRC 校验的计算并恢复列表数据项。

状态机操作：Coremark 中状态机相关的实现在 core_state.c 文件中，在 main 函数中检测到需要进行状态机测试的时候，调用 core_bench_state()函数来完成状态机测试，测试函数为 core_bench_state()，状态机的实现函数为 core_state_transition()，core_state_transition()的函数实现与嵌入式中常采用的状态机实现一致，即使用 for/while 来做循环控制，使用 switch case 来做状态迁移控制。

表 1-1CoreMark 文件

文件	功能
core_main.c	主程序
core_state.c	状态机相关函数
core_list_join.c	列表相关函数
core_matrix.c	矩阵相关函数
core_util.c	CRC 相关函数
coremark.h	配置定义、开关及相关数据结构

1.3 Coremark 测试结果

CoreMark 输出结果为 Iteration/Sec，即每秒钟跑了多少次 coremark 测试（其中测试时间不少于 10s）。这和 MCU 主频有关，所以我们评判一个 MCU 性能时，会将其对齐到每 MHZ，即我们看到的更多的是 Iteration/Sec/MHZ，即最后需要的结果 CoreMark/Mhz。

对比整体性能使用 Iterations/Sec

对比不同 MCU 性能使用 coremark/Mhz

例如 CH32H417 Qingke V5 内核在 400Mhz 主频下，ITCM 中运行 Coremark 跑分是 2720.853897，每 MHZ 结果为 $2720.853897 / (400/1) = 6.802134$ coremark/Mhz。

图 1-1CH32H417 V5_CoreMark 结果

```
MCU: CH417
V5SystemClk: 400000000Hz
2K performance run parameters for coremark.
CoreMark Size      : 666
Total ticks        : 850476204
Total time (secs): 12.863608
Iterations/Sec     : 2720.853897
Iterations         : 35000
Compiler version   : GCC15.2.0
Compiler flags     : -Ofast
Memory location    : Static
seedcrc           : 0xe9f5
[0]crclist        : 0xe714
[0]crcmatrix      : 0x1fd7
[0]crcstate       : 0x8e3a
[0]crcfinal       : 0x4983
Correct operation validated. See README.md for run and
reporting rules.
CoreMark 1.0 : 2720.853897 / GCC15.2.0 -Ofast / Static
If working at 1MHz, CoreMark 1.0 : 6.802134/MHz
```

第2章 Coremark 影响因素

2.1 编译器/优化等级

CoreMark 和编译器息息相关。编译器负责将 C 代码翻译成机器指令，而不同编译器、不同优化选项生成的指令在效率和顺序上天差地别。

2.1.1 优化选项的直接影响

除了编译器本身，传给编译器的“参数”（优化选项）也至关重要。现代编译器（如 GCC）提供了不同的优化级别，如 -O1、-O2、-O3、-Ofast 等。优化级别越高，编译器就越激进地尝试用更快的指令替换原有逻辑。

性能提升：通常情况下，开启更高级别优化能显著提升 CoreMark 分数。例如，CH32V307 启用 -O1，跑分达到 319.084549。CH32V307 启用 -Ofast，跑分达到 469.522753。

图 2-1CH32V307 不同优化选项 CoreMark 结果

MCU: CH32V307	MCU: CH32V307
SystemClk: 144000000Hz	SystemClk: 144000000Hz
2K performance run parameters for coremark.	2K performance run parameters for coremark.
CoreMark Size : 666	CoreMark Size : 666
Total ticks : 2256455234	Total ticks : 1533472011
Total time (secs): 15.669828	Total time (secs): 10.649111
Iterations/Sec : 319.084549	Iterations/Sec : 469.522753
Iterations : 5000	Iterations : 5000
Compiler version : GCC15.2.0	Compiler version : GCC15.2.0
Compiler flags : -O1	Compiler flags : -Ofast
Memory location : Static	Memory location : Static
seedcrc : 0xe9f5	seedcrc : 0xe9f5
[0]crclist : 0xe714	[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7	[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a	[0]crcstate : 0x8e3a
[0]crcfinal : 0xbd59	[0]crcfinal : 0xbd59
Correct operation validated. See README.md for run and reporting rules.	Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 319.084549 / GCC15.2.0 -O1 / Static	CoreMark 1.0 : 469.522753 / GCC15.2.0 -Ofast / Static
If working at 1MHz, CoreMark 1.0 : 2.215865/MHz	If working at 1MHz, CoreMark 1.0 : 3.260575/MHz

沁恒通过设置 GCC 编译器的命令行选项 -funroll-all-loops -finline-limit=600 -ftree-dominator-opts -fno-if-conversion2 -fselective-scheduling -fno-code-hoisting，来控制编译器优化 C 代码。

表 2-1GCC 编译器设置

编译选项	作用
-funroll-all-loops	循环展开：将循环体内的代码复制多次，以减少或消除循环控制（如判断循环条件、跳转）的开销。
-finline-limit=600	函数内联：当函数代码大小小于 600 个单位时，编译器会尝试将函数调用处直接替换为函数本身的代码，避免调用和返回的开销。
-ftree-dominator-opts	支配树优化：一种高级的代码分析技术，用于消除冗余计算，比如将循环内结果不变的表达式提到循环外面。

-fno-if-conversion2	禁用第二次条件转换： 不让编译器将某些条件分支（如 if 语句）转换为算术指令或条件执行指令。
-fselective-scheduling	选择性指令调度： 编译器会尝试在不改变程序逻辑的前提下，重新排列指令的顺序，以更好地利用 CPU 的多个执行单元，减少“流水线停顿”。
-fno-code-hoisting	禁用代码上提： 不让编译器将循环内不变的计算提前到循环外面执行。这与-ftree-dominator-opts 的效果有点像，但更激进

Coremark 是一个计算密集型、循环多、函数调用频繁的测试代码。

-funroll-all-loops 把循环打开，减少分支跳转，CPU 流水线跑的更满。

-finline-limit=600 把小函数内联，消除函数调用开销。

-fselective-scheduling 重排指令隐藏内存访问延迟。

2.1.2 编译器版本差异

即使都是 GCC，不同版本的性能也有区别。随着新版本的发布，编译器会加入对新架构的更好支持和新优化技术。例如，在测试 CH32V307 芯片时，GCC 15 相比 GCC 12 版本，在 CoreMark 跑分上就有 1.5% 的提升。

图 2-2CH32V307 不同编译器版本 Coremark

<pre>MCU:CH32V307 SystemClk:144000000Hz 2K performance run parameters for coremark. CoreMark Size : 666 Total ticks : 1533472009 Total time (secs): 10.649111 Iterations/Sec : 469.522753 Iterations : 5000 Compiler version : GCC15.2.0 Compiler flags : -Ofast Memory location : Static seedcrc : 0xe9f5 [0]crclist : 0xe714 [0]crcmatrix : 0x1fd7 [0]crcstate : 0x8e3a [0]crcfinal : 0xbd59 Correct operation validated. See README.md for run and reporting rules. CoreMark 1.0 : 469.522753 / GCC15.2.0 -Ofast / Static If working at 1MHz, CoreMark 1.0 : 3.260575/MHz</pre>	<pre>MCU:CH32V307 SystemClk:144000000Hz 2K performance run parameters for coremark. CoreMark Size : 666 Total ticks : 1556918804 Total time (secs): 10.811936 Iterations/Sec : 462.451862 Iterations : 5000 Compiler version : GCC12.2.0 Compiler flags : -Ofast Memory location : Static seedcrc : 0xe9f5 [0]crclist : 0xe714 [0]crcmatrix : 0x1fd7 [0]crcstate : 0x8e3a [0]crcfinal : 0xbd59 Correct operation validated. See README.md for run and reporting rules. CoreMark 1.0 : 462.451862 / GCC12.2.0 -Ofast / Static If working at 1MHz, CoreMark 1.0 : 3.211471/MHz</pre>
--	--

2.2 内存系统配置

将代码加载到 RAM（ITCM）可大幅提升 Coremark，因为避免了 Flash 访问延迟。

RAM：访问速度极快，通常与 CPU 同速，可以实现零等待。

Flash：物理特性决定了它的读取速度比 RAM 慢很多。当 CPU 频率超过 Flash 的极限时，就需要插入等待周期。例如：当 HCLK 超过 160MHz 时，CPU 访问 flash 时需要插入 4 个等待周期。CH32V205 在 160M 情况下运行 Coremark 跑分为 2.248/Mhz。CH32V205 在 40M 情况下运行 Coremark 跑分为 3.561/Mhz。

图 2-3CH32V205 CoreMark 不同内存配置

MCU:CH32V205	MCU:CH32V205
SystemClk111:40000000Hz	SystemClk111:160000000Hz
FLASH->ACTLR:00000010	FLASH->ACTLR:00000033
2K performance run parameters for coremark.	2K performance run parameters for coremark.
CoreMark Size : 666	CoreMark Size : 666
Total ticks : 842354388	Total ticks : 1778670849
Total time (secs): 21.058859	Total time (secs): 11.116692
Iterations/Sec : 142.457855	Iterations/Sec : 359.819243
Iterations : 3000	Iterations : 4000
Compiler version : GCC15.2.0	Compiler version : GCC15.2.0
Compiler flags : -Ofast	Compiler flags : -Ofast
Memory location : Static	Memory location : Static
seedcrc : 0xe9f5	seedcrc : 0xe9f5
[0]crclist : 0xe714	[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7	[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a	[0]crcstate : 0x8e3a
[0]crcfinal : 0xcc42	[0]crcfinal : 0x65c5
Correct operation validated. See README.md for run and reporting rules.	Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 142.457855 / GCC15.2.0 -Ofast / Static	CoreMark 1.0 : 359.819243 / GCC15.2.0 -Ofast / Static
If working at 1MHz,CoreMark 1.0 : 3.561446/MHz	If working at 1MHz,CoreMark 1.0 : 2.248870/MHz

历史版本

日期	版本	变更内容
2026/7/8	V1.0	初版发行

声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。